# Government Data APIs

Week 11 · APIs module · Textbook Chapter 11

**Brian C. Keegan, Ph.D.**
Associate Professor, Department of Information Science
University of Colorado Boulder

October 26, 28 & 30, 2026

# This week at a glance

Three public APIs, each showing one idea that holds for every API. This week, every class is live coding, from one lab notebook in three parts.

**Monday** | Lab, part 1: the Census

- Keys, and *what can I ask for?* — the variable and geography catalogs

**Wednesday** | Lab, part 2: more of the Census, then FRED

- Margins of error and missing estimates; then *let the server compute*

**Friday** | Lab, part 3: more of FRED, then the EIA

- *Let the server filter*; then textbook revisions

## The lab

`labs/week-11/lab-11.ipynb` in the course repository. Each part is added before its day.

## Three keys, as environment variables

`CENSUS_KEY` Monday and Wednesday; `FRED_KEY` Wednesday and Friday; `EIA_KEY` Friday. Links are in the Canvas announcement and the lab's first cell.

## Due Sunday, Nov 1

Chapter 11's **Recommended Exercises**, from its companion notebook. Graded on good-faith attempt to complete rather than getting everything working.

# 1. Monday · Live-Coding Lab

# This week, every class is live coding

## Find it | `labs/` in the course repository

1. Open the **labs** folder of `cuinfoscience/INF04617-Fall2026` on GitHub
2. Open `week-11`, click `lab-11.ipynb`, then **Download raw file**
3. Open it in JupyterLab in the `webdata` environment, *after* setting your keys

## Use it | one part a day

- I write the code on screen, and you write it in your copy
- Put **your** email in the User-Agent in Step 0
- Today: Part 1, Steps 0–6, about 50 minutes
- Wednesday and Friday: download the new copy and start at that part's setup step

## Solutions

`lab-11-key.ipynb`, beside it: every cell filled in and run. Fall behind? Copy the cell from the key and keep going.

## Nothing to submit

The lab is practice. After each step, a **Discuss** box holds questions we'll talk over before the next.

## Error flavors

A missing key stops Step 0 *on purpose*: set it and run again. A cell that fails with your key set is *broken*: tell me, or open an issue.

## No key yet?

Follow along: the key shows every output. A Census key works only after you click the link in its email.

# Public data by mandate

Government APIs exist because of democratic commitments to **transparency** — not business strategy:

- The **Census Bureau** is constitutionally mandated to count the population.
- The **Federal Reserve** publishes the economic data it collects; FRED is the St. Louis Fed's service for it.
- The **Energy Information Administration** was created in 1977, after the energy crises, to publish independent energy statistics.
- The **Library of Congress** publishes Congress's legislative record at Congress.gov.

They embody the **openness** value from Ch. 3, *The Post-API Age*: public data, freely accessible by mandate rather than platform goodwill.

### Stable, but less polished

Agencies don't pivot business models or get acquired — so these APIs are unusually **stable**. The tradeoff: dense docs, cryptic variable names, unhelpful errors. Navigating that *is* the skill.

## Three APIs, three ideas

Week 10's Wikipedia taught **pagination**. Each API this week teaches one more idea you'll meet in every API:

| Day | API | The idea |
|---|---|---|
| Monday, Wednesday | Census Bureau | **Discovery**: ask the API what you can ask for |
| Wednesday, Friday | FRED | **Let the server compute**: transforms in the request |
| Friday | EIA | **Let the server filter**: facets narrow the data before it's sent |
| The chapter | Congress.gov | **Follow the links**: records point to related records |

And one idea across all of them: a **key** says who you are, and it stays out of your notebook.

# Keys stay out of the notebook

A **key** identifies you, counts your requests, and enforces rate limits. A key typed into a notebook travels with it: into a commit, a submission, a classmate's laptop.

Store it with the `webdata` environment, once, in a terminal (on Windows, the Anaconda Prompt); then restart Jupyter and read it:

```
conda env config vars set CENSUS_KEY="your-key" -n webdata
```

```
import os
census_key = os.environ["CENSUS_KEY"] # a missing key stops here
```

The User-Agent still says who sent each request: `WebDataScience/1.0 (your-email@colorado.edu)`.

## Or in your shell

`export CENSUS_KEY="your-key"` in `~/.zshrc` or `~/.bashrc`; `setx` on Windows. Chapter 11's "Setting Environment Variables" shows both ways.

## Or a file

A JSON file, `api_keys.json`, added to `.gitignore` *before your first commit*. A key in git history stays there even after you delete the file.

## Names

The lab uses `CENSUS_KEY`, `FRED_KEY`, and `EIA_KEY`. Chapter 11 uses the same names, and `DATAGOV_KEY` for Congress.gov.

# Discovery: two catalogs before any data

## **What** | the variable catalog

- Variables come in **groups**: `B25034` is "Year Structure Built"
- `B25034_002E`: group, line, and `E` for estimate (`M` is its margin of error)
- `.../acs5/groups/B25034.json` is itself an API

## **Where** | the geography catalog

- `.../acs5/geography.json` lists every level the dataset answers for
- Each says what it **requires**: a `place` needs a `state`; a `tract` needs a `state` and a `county`
- Places are **FIPS codes**, as strings: Colorado `08`, Boulder `07850`

Codes change between survey years: learn to look them up, not to memorize them. Then **sanity-check** one value: Boulder's median household income is $87,493 (±$6,221), not 7 or 70 million.

# Census — from the catalogs to a real place

Name the variables with `get`, the place with `for`, and what it requires with `in`:

```
census_base = "https://api.census.gov/data/2024/acs/acs5"
params = {"get": "NAME,B25034_001E,B25034_002E,B25034_003E",
          "for": "place:07850", # Boulder: a FIPS code, kept as a string
          "in": "state:08", # a place requires its state
          "key": census_key}
data = requests.get(census_base, params=params, headers=HEADERS).json()

df = pd.DataFrame(data[1:], columns=data[0]) # row 0 is the header
numeric_cols = [c for c in df.columns if c.startswith("B25034")]
df[numeric_cols] = df[numeric_cols].apply(pd.to_numeric) # values arrive as strings
```

Boulder has 47,465 housing units; 646 were built in 2020 or later, 3,836 in the 2010s.

## 2. Wednesday · Live-Coding Lab

# Today: Part 2, more of the Census, then FRED

We finish what Monday started, then meet a second API:

- **The Census**: compare two places, read a **margin of error**, and find the places the survey couldn't measure
- **FRED** (Federal Reserve Economic Data, from the St. Louis Fed) **computes** before it answers: changes and percent changes. Today's idea: *let the server compute*

**Use it** | Steps 4–9, about 47 minutes

- Download the new `lab-11.ipynb`; start at **Step 4**, which imports and checks the keys again
- Put **your** email in the User-Agent

## Two keys today

`CENSUS_KEY` and `FRED_KEY`. FRED calls the parameter `api_key`; the Census Bureau calls it `key`.

## No key yet?

FRED's comes with a free account. Follow along from the key until you have it.

## Friday

More of FRED, then the EIA. Congress.gov and the chapter's join of two agencies' data are there to read on your own.

# `KeyError`: Python can't see the key

Step 4 checks before it reads, and never prints the key:

```
for name in ["CENSUS_KEY", "FRED_KEY"]:
    print(name + ":",
          "set" if name in os.environ else "MISSING")
# CENSUS_KEY: set
# FRED_KEY: set
```

Jupyter sees only the variables set **before it started**, in **the place it started from**. On Monday, `export` didn't reach it when:

- Jupyter was already running, or was started from another terminal
- Jupyter was started from Anaconda Navigator, which, on a Mac, doesn't read `~/.zshrc`
- on Windows, `setx` reaches only prompts opened after it

## The dependable fix

Store the key with the `webdata` environment, in a terminal (on Windows, the Anaconda Prompt); then quit Jupyter and start it again:

```
conda env config vars set FRED_KEY="your-key" -n webdata
```

## Can't restart now?

The step's last cell asks for the key, and hides what you paste. It lasts until the kernel restarts:

```
from getpass import getpass
fred_key = getpass("Paste your FRED key: ")
```

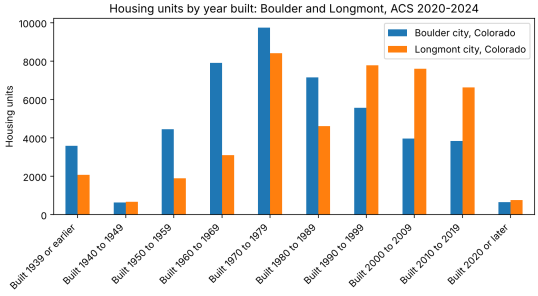
# Two cities, ten decades

Step 5 puts two places in one request, asks for all ten decades of `B25034`, and labels them from Monday's catalog:

```
housing_vars = [f"B25034_{line:03d}E"
                for line in range(2, 12)]
params = {"get": "NAME," + ",".join(housing_vars),
          "for": "place:07850,45970",
          "in": "state:08", "key": census_key}

housing_catalog = requests.get(
    f"{census_base}/groups/B25034.json",
    headers=HEADERS).json()["variables"]
labels = {var: housing_catalog[var]["label"]
              .split("!!")[-1] for var in housing_vars}
```

Boulder's biggest decade was the 1970s; Longmont has more units from every decade since the 1990s.




Drawn by the chapter's code from the Census Bureau's API, 2026-10-25 (chapter 11, figure 11.5).

## Microsprint

Counts don't compare cities of different sizes; shares do. Built in 2000 or later: Castle Rock 71%, Fort Collins 37%, Longmont 34%, Lakewood 18%, Boulder 18%.

# Check a number against its margin

Each estimate has a margin of error, the same name ending in `M`, at 90% confidence. Step 6 asks for both:

```
params = {"get": "NAME,B19013_001E,B19013_001M",
          "for": "place:07850,45970",
          "in": "state:08", "key": census_key}
...
incomes["low"] = incomes["B19013_001E"] - incomes["B19013_001M"]
incomes["high"] = incomes["B19013_001E"] + incomes["B19013_001M"]
```

| City | Estimate | Margin | Low | High |
|---|---|---|---|---|
| Boulder | 87,493 | 6,221 | 81,272 | 93,714 |
| Longmont | 90,671 | 2,975 | 87,696 | 93,646 |

## The ranges overlap

From \$87,696 to \$93,646, both are possible. The survey can't say which city's households earn more.

## Discuss

Boulder's margin is twice Longmont's. What about the city could make its estimate less certain?

Which checks like this one could a pipeline run on its own, every time it runs?

# When there's no estimate: `-666666666`

Step 7 asks for every place in Colorado, `for=place:*` with `in=state:08`, and looks before it trusts:

```
places["income"].describe().round()
# count 482.0
# mean -114723934.0
# min -666666666.0
# max 250001.0
```

83 of the 482 places have no median-income estimate: too few households answered. The API doesn't leave the value out; it sends a number that can't be an income. (250,001 is a cap: "$250,000 or more".)

| Place | People | Median income |
|---|---|---|
| Boulder | 106,433 | 87,493 |
| Campo | 58 | 53,125 (±40,478) |
| Bonanza | 20 | `-666666666` |
| Carbonate | 0 | `-666666666` |

## Average it as is

The mean of those four is −333,298,178. Mask the codes first, `towns.mask(towns < 0)`, and it's 70,309.

## What the codes mean

The Census Bureau's notes on estimate and annotation values list each one; a margin of error of `-222222222` goes with it.

## Discuss

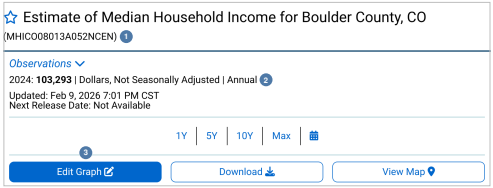
Carbonate has no residents, yet it has a row. What does the Census count as a place?

# FRED: find the series, then ask what it is

Every one of FRED's 800,000 series has an **ID** (1), on its page. The `series` endpoint describes it; `series/observations` sends its values:

```
fred_base = "https://api.stlouisfed.org/fred"
params = {"series_id": "MHICO08013A052NCEN",
          "api_key": fred_key, "file_type": "json",
          "observation_end": "2024-12-31"}
data = requests.get(f"{fred_base}/series/observations",
                    params=params, headers=HEADERS).json()
income = pd.DataFrame(data["observations"])[["date", "value"]]
income["value"] = pd.to_numeric(income["value"],
                                errors="coerce") # "." is missing
```

2024: $103,293 (2) — not the Census API's $103,994. Different programs: name the one you cite.




Boulder County's median household income on FRED, captured 2026-10-25 (chapter 11, figure 11.2).

# Let the server compute: `units`

Step 9 gathers Step 8's lines into `fred_observations(series_id, **params)`. Then the *same series* twice more, one parameter changed:

```
change = fred_observations("MHICO08013A052NCEN",
           units="chg", observation_end="2024-12-31")
growth = fred_observations("MHICO08013A052NCEN",
           units="pch", observation_end="2024-12-31")
```

| Year | Dollars | Change | Percent change |
|---|---|---|---|
| 2022 | 96,584 | 6,257 | 6.9 |
| 2023 | 97,170 | 586 | 0.6 |
| 2024 | 103,293 | 6,123 | 6.3 |

## What `units` can be

| | |
|---|---|
| `lin` | as published |
| `chg` | change |
| `ch1` | change from a year before |
| `pch` | percent change |
| `pc1` | percent change from a year before |
| `log` | natural logarithm |

## Microsprint

In which years did the income *fall*? Six, from 2001 to 2020; the largest, $3,003 in 2009.

## 3. Friday · Live-Coding Lab

# Today: Part 3, more of FRED, then the EIA

**Lab** | Steps 10–15, about 35 minutes

- **FRED** computes one more thing: a lower **frequency**, and inflation from it; then Denver against the nation
- The **EIA** (U.S. Energy Information Administration) **filters**: one state, one sector, one column, one span of years. Today's idea: *let the server filter*
- Download the new `lab-11.ipynb`; start at **Step 10**, which sets up again

**Then** | the textbook, 15 minutes

- What the lab turned up in chapter 11, as a pull request

## Two keys today

`FRED_KEY` and `EIA_KEY`. A `KeyError`? Wednesday's fix: `conda env config vars set`, then restart Jupyter.

### No key yet?

The EIA's arrives by email within minutes. Follow along from the key until it does.

### In the chapter, not the lab

Congress.gov's links, paging with `offset` and `next`, and the join of the Census Bureau's incomes with the EIA's electricity bills.

# Let the server compute: `frequency`, then `units`

One series, `CPIAUCNS` (the national Consumer Price Index, monthly), three requests:

```
years = {"observation_start": "2015-01-01",
         "observation_end": "2025-12-31"}
monthly = fred_observations("CPIAUCNS", **years)
annual = fred_observations("CPIAUCNS", frequency="a",
           aggregation_method="avg", **years)
inflation = fred_observations("CPIAUCNS", frequency="a",
           aggregation_method="avg", units="pc1", **years)
```

FRED lowers the frequency first, then applies the units: each year's mean of twelve months, then its percent change from the year before.

## Trust, but check once

Microsprint: the same numbers from `monthly`, with `groupby()` and `pct_change()`.

| Year | FRED | pandas |
|---|---|---|
| 2023 | 4.1 | 4.1 |
| 2024 | 2.9 | 2.9 |
| 2025 | 2.6 | 2.6 |

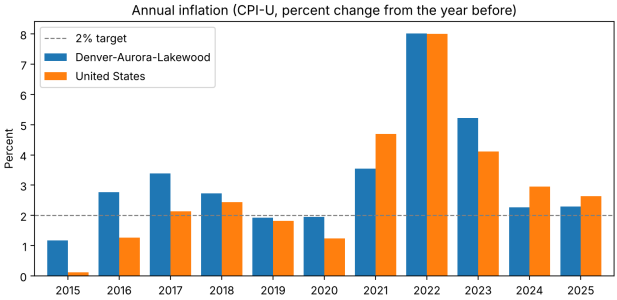
Why can FRED give 2015's inflation, and pandas can't?

# Denver against the nation

The Bureau of Labor Statistics' CPI for the Denver area, `CUUSA433SA0`, is annual already: it needs only `units="pc1"`.

```
denver = fred_observations(
    "CUUSA433SA0", units="pc1", **years)
cpi = denver.merge(inflation, on="date",
    suffixes=("_denver", "_national"))
```

Denver ran ahead from 2015 to 2020, and behind since 2024. Housing is the largest part of the CPI: what would you ask FRED next?




Drawn by the chapter's code from FRED, 2026-10-25 (chapter 11, figure 11.6). The dashed line is the Federal Reserve's 2% target.

# EIA: routes that describe themselves

The EIA's API is a tree of **routes**. Ask for one without `data/`, and it says what it holds:

```
eia_base = "https://api.eia.gov/v2"
meta = requests.get(f"{eia_base}/electricity/retail-sales/",
                    params={"api_key": eia_key},
                    headers=HEADERS).json()["response"]
print([f["id"] for f in meta["facets"]])
# ['stateid', 'sectorid']
for column, about in meta["data"].items():
    print(column, "|", about["units"])
# price | cents per kilowatt-hour ...and 3 more
```

A **facet** filters the rows; a **data column** (1) is what each row holds. Monday's idea, *what can I ask for?*, from a different API: here the route describes itself, where the Census Bureau's catalog was `groups.json`.




The EIA's API Dashboard, captured 2026-10-25 (chapter 11, figure 11.3): its menus (2) are those routes, and it writes the request (3).

# Let the server filter

Add `data/` to the route, and each parameter narrows the rows. The brackets are part of the names:

```
params = {"frequency": "annual", "data[]": "price",
          "facets[stateid][]": "CO",
          "facets[sectorid][]": "RES", # homes
          "start": "2001", "end": "2025",
          "sort[0][column]": "period",
          "sort[0][direction]": "asc",
          "api_key": eia_key}
prices = requests.get(f"{eia_base}/electricity/retail-sales/data/",
            params=params, headers=HEADERS).json()["response"]
print("Rows:", prices["total"]) # Rows: 25
```

25 rows, from a route that holds every state and sector by month, quarter, and year. Colorado's homes paid 15.69 cents a kilowatt-hour in 2025.

## Microsprint

Change one facet: a sector (`COM`, `IND`) or a state (`UT`, `NM`). Colorado's businesses paid 12.37 cents in 2025.

## Still strings

Prices and `total` arrive as text, like the Census Bureau's numbers: `pd.to_numeric()` before any math.

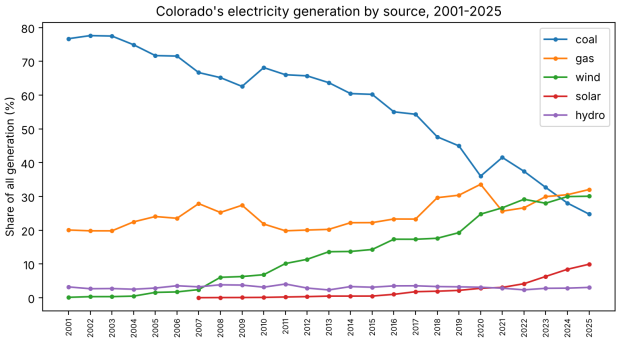
## Filter before you page

The EIA sends at most 5,000 rows a request. A facet, a shorter span, or a lower frequency may fit the answer in one; the chapter's "Paging with `offset` and `length`" covers the rest.

# What a filtered request can show

The chapter asks the same API for Colorado's annual generation from five sources, plus the total: one facet, six values, one request.

Coal made 76.7% of the state's electricity in 2001 and 24.7% in 2025. Wind grew from almost nothing to 30.0%; 2024 was the first year coal wasn't the largest source.




Drawn by the chapter's code from the EIA's API, 2026-10-25 (chapter 11, figure 11.7).

# Friday: from the lab to the textbook (15 minutes)

Look back at what the lab turned up in chapter 11, and propose one change as a **pull request**:

- **A cell that broke.** On Monday, `os.environ["CENSUS_KEY"]` stopped with a `KeyError` for some of us after `export`. Does "Setting Environment Variables" say why, and what to do?
- **A sentence that confused.** Which step needed the website beside the API to make sense? Does the chapter's tour do that for the Census table, and could it do more?
- **An output that has changed.** The EIA's 2025 figures are preliminary. Compare Step 14's prices with the chapter's.
- **A microsprint that would make a good exercise.** Searching `groups.json` for your own topic (Step 1), or two cities' margins (Step 6).

## How to propose one

The course's `handouts/common/revision-framework.md` says what each kind of change needs to be accepted.

## A good review

Run the change; name the line; separate "must fix" from "nice to have"; end with a verdict.

## Key takeaways

1. Government APIs serve **public mandates**: the census, the Federal Reserve's data, the EIA's statistics.
2. Each carried one idea you'll meet again: **ask what you can ask for** (Census), **let the server compute** (FRED), **let the server filter** (EIA).
3. **Start from the website**: find the number there, then ask the API for the same number, and check that they match.
4. **Read what isn't a number**: a margin of error, a code like `-666666666`, a cap at $250,000, a `"."` for missing.
5. **Keys stay out of notebooks**: environment variables, checked with `set` or `MISSING`, never printed.